

AY 2025/2026



POLITECNICO
MILANO 1863

Internet Of Things: Challenge 3

Part 1

Alberto Biffi - 10677341

Matteo CESANA Professor
Antonio BOIANO Fabio PALMESE

Version 1.0
May 11, 2026

Contents

1	Introduction	2
2	Part 1: Challenge – Node Red	3
2.1	Process the CSV file in Node-Red	3
3	Challenge: What to do? (1) 1 point	4
4	What to do? (2) 2 points	5
5	What to do? (3) 2 points	11
6	Testing	15
7	Deliveries	15

1 Introduction

This is the flow that I've used to answer all the points in the challenge. After I've provided an overview of each part.

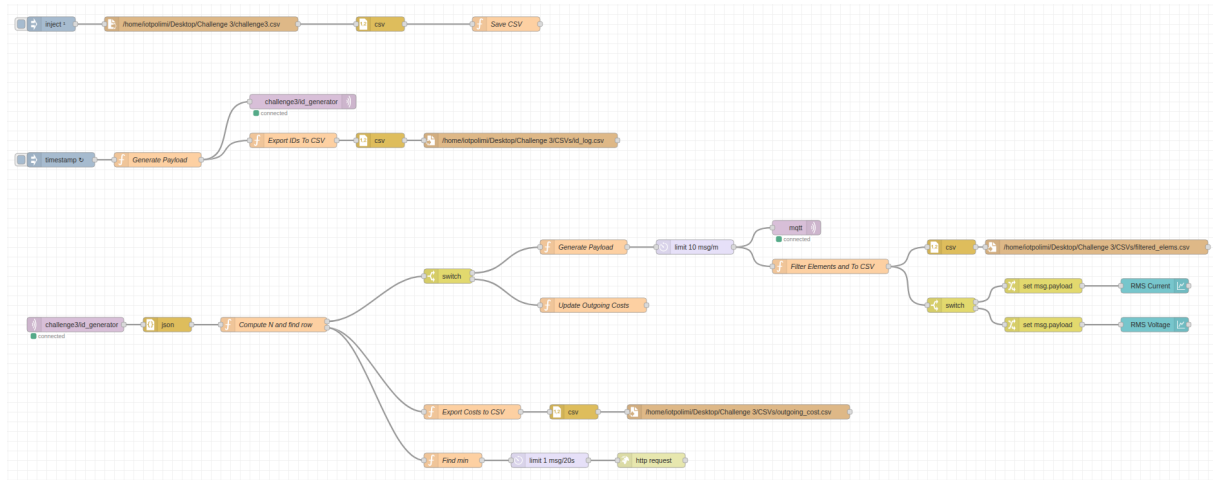


Figure 1: Overview

ThingSpeak Channel ID **3368636**

2 Part 1: Challenge – Node Red

2.1 Process the CSV file in Node-Red

I've decided to preload the CSV in memory at the start.

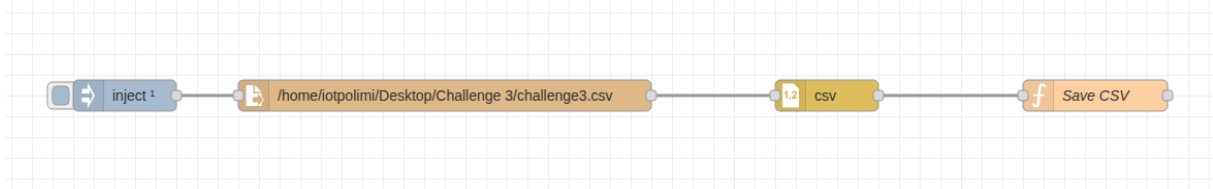


Figure 2: Preload CSV

The inject is set to start only once at the start of the program. Then, the Save CSV function creates a dictionary with all of the rows.

```
1 let csvDict = {};  
2  
3 for (let i = 0; i < msg.payload.length; i++) {  
4   let row = msg.payload[i];  
5   let packetNum = parseInt(row["Packet Number"]);  
6   csvDict[packetNum] = row;  
7 }  
8  
9 flow.set("csvData", csvDict);  
10  
11 return msg;
```

Listing 1: Save CSV

3 Challenge: What to do? (1) 1 point

- Create a flow to periodically publish MQTT messages to the local Mosquitto broker (localhost, port 1884), to the topic challenge3/id_generator (be sure to start the Mosquitto broker locally with the correct port).
- Messages should be sent with a rate of 1 message every 1 second.
- Each message should contain in the payload a JSON-formatted string with a random number (id) between 0 and 30000, and the time at which the message is generated (UNIX timestamp).
- When sending the message, also save its fields in a CSV (id_log.csv) in the form: No.,ID,TIMESTAMP where No. is the incremental row number.

First, we set the Mosquitto Broker Port to 1884, adding these two rows to the mosquitto.conf file.

```
1 listener 1884
2 allow_anonymous true
```

Listing 2: Change Mosquitto Port

Now let's analyze our flow.

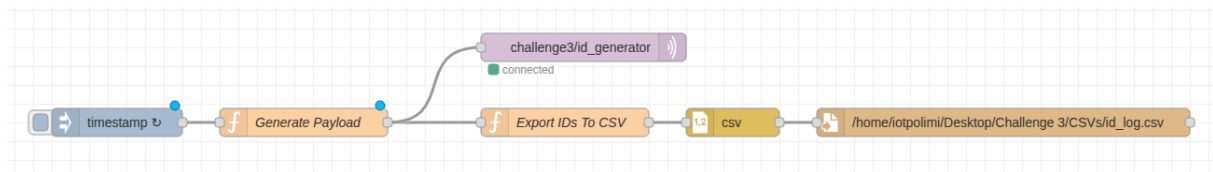


Figure 3: Publisher Flow

We set the inject node to repeat the injection every second. Saving the timestamp each time in the msg.payload.

The Generate Payload function is used to create the required payload that we have to send to the local Mosquitto broker.

```
1 var timestamp = msg.payload;
2 var id = Math.floor(Math.random() * 30001);
3
4 msg.payload = {
5   "id": id,
6   "timestamp": timestamp
7 };
8
9 return msg;
```

Listing 3: Generate Payload

It generates a value between 0 and 30000 and saves it in the payload with the timestamp. Now the flow is split in two. One will send the message to the local Mosquitto broker, and the other part will generate the id_log.csv file.

The Export IDs To CSV function will format the payload into CSV format.

```
1 let rowN = flow.get("rowCounter") || 1;
2
3 flow.set("rowCounter", rowN + 1);
4
5 msg.payload = {
6   "No.": rowN,
7   "ID": msg.payload.id,
8   "TIMESTAMP": msg.payload.timestamp
9 };
10
11 return msg;
```

Listing 4: Export IDs To CSV

4 What to do? (2) 2 points

In another branch of the flow (same flow):

- Subscribe to the topic challenge3/id_generator in the local broker (localhost, port 1884)
- After receiving a message from the subscription, take the ID and compute the remainder after division by 5218 to get N
- For every message received, process the challenge3.csv file and take the message whose packet number is equal to the received identifier N

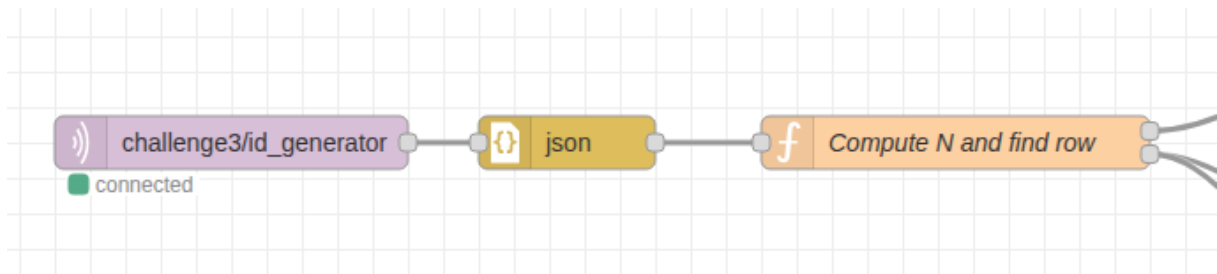


Figure 4: Subscriber

We use the MQTT In node to subscribe to the topic required and the JSON node to parse the payload received. Then the Compute N and find row function node will contain the logic for stopping and calculating the value of N. We needed 2 different outputs for this node so that we can split what to do when we finish receiving the messages.

```
1 var n = msg.payload.id % 5218;
2
3 let countReceived = flow.get("countReceived") || 1;
4
5 if(countReceived > 201){
6     return [null, null];
7 }
8
9 if(countReceived == 201){
10     flow.set("countReceived", countReceived + 1);
11     return [null, msg];
12 }
13
14 flow.set("countReceived", countReceived + 1);
15
16 var n = msg.payload.id % 5218;
17 let csvDict = flow.get("csvData");
18
19 if (csvDict && csvDict[n]) {
20     let targetRow = csvDict[n];
21
22     msg.csvRow = targetRow;
23     msg.originalID = msg.payload.id;
24     msg.n = n;
25
26     return [msg, null];
27 } else {
28     return [null, null];
29 }
```

Listing 5: Compute N and Find Row

Firstly, we calculate the value N, then we initialize the counter of the received messages. We use that counter for managing the outputs.

When the counter is < 200 , we will create the payload containing the correct row selected from the previously loaded csv file.

Otherwise, when we have received our 200th message, we will send the output only on the second exit to create the csv and send the message to our ThingSpeak channel. I'll explain those in detail later.

- If the message with Packet No. = N in the file contains (under “Command String” column) a ZBEE_ZCL Layer, perform an MQTT Publish to the topic named after the value in “Device Name ZigBee Source” corresponding column.
- All the addresses must be in hexadecimal format e.g. 0x123ab
- Limit the messages published in this step to a rate of ten messages per minute.
- In addition, after sending the publish message, if the Command String contains an RMS Current or an RMS Voltage attribute or an Active Power attribute, save selected elements in a CSV (filtered_elems.csv).
- Finally plot the RMS Current and RMS Voltage associated value (e.g. 0 for RMS Current and 223 for RMS Voltage) in two Node-Red charts one for RMS Voltage and one for RMS Current.

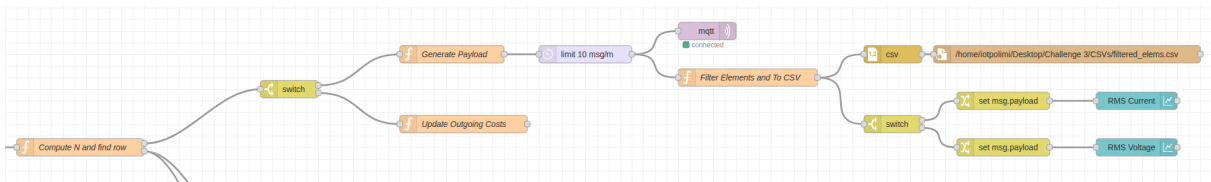


Figure 5: ZBEE_ZCL Switch

We have a switch node that will look at the `msg.csvRow["Command String"]` and select only the messages containing "ZBEE_ZCL" in that string.

The other exit, as we will see later, is used for the messages containing the "Link Status". The messages containing the required layer will be used by the Generate Payload function node to create the payload to send.

```

1 let row = msg.csvRow;
2
3 msg.topic = "ZigBee/" + row["Device Name ZigBee Source"];
4
5 msg.payload = {
6   "timestamp": Date.now(),
7   "id": msg.originalID,
8   "wpan.src": row["Source Address"],
9   "wpan.dst": row["Destination Address"],
10  "zbee.src": row["Source Address ZigBee"],
11  "zbee.dst": row["Destination Address ZigBee"],
12  "topic": msg.topic,
13  "payload": row["Command String"]
14 }
15
16 return msg;

```

Listing 6: Generate Payload

This message will then go to a delay node that will limit the output ratio to 10 messages per minute. The message exiting this node will go to the MQTT Out node and also the Filter Elements and To CSV function node.

The MQTT Out node will not specify the topic because it will be passed through the `msg.topic` variable.

For creating the `filtered_elem.csv`, we will use the Filter Elements and To CSV function node.

```

1 let cmdStr = msg.csvRow["Command String"];
2
3 if (cmdStr.includes("RMS Current") || cmdStr.includes("RMS Voltage") ||
4   cmdStr.includes("Active Power")) {
5   try {
6     let data = JSON.parse(cmdStr.replace(/'/g, ''));
7
8     let seqNum = data["Sequence Number"];
9     let attributes = [].concat(data["Attribute"] || []);
10    let statuses = [].concat(data["Status"] || []);
11    let dataTypes = [].concat(data["Data Type"] || []);
12
13    let allValues = [];
14    if (data["Int16"]) allValues = allValues.concat(data["Int16"]);
15    if (data["Uint16"]) allValues = allValues.concat(data["Uint16"]);
16
17    let msgList = [];
18
19    for (let i = 0; i < attributes.length; i++) {
20      let attrName = String(attributes[i]).split(" ")[0];
21
22      if (attrName === "RMS Current" || attrName === "RMS Voltage"
23        || attrName === "Active Power") {
24
25        if (statuses[i] === undefined || dataTypes[i] ===
26          undefined || allValues[i] === undefined) {
27          continue;
28        }
29
30        let filterCount = flow.get("filteredCount") || 1;
31        flow.set("filteredCount", filterCount + 1);
32
33        let statusVal = String(statuses[i]).split(" ")[0];
34        let typeVal = String(dataTypes[i]).split(" ")[0];
35        let rawValue = String(allValues[i]).split(" ")[0];
36
37        let newMsg = {
38          payload: {
39            "No.": filterCount,
40            "Timestamp": msg.payload.timestamp || msg.
41              timestamp,
42            "Sequence Number": seqNum,
43            "Attribute": attrName,
44            "Status": statusVal,
45            "Data Type": typeVal,
46            "Data Value": rawValue
47          },
48          chartValue: Number(rawValue),
49          chartType: attrName
50        };
51        msgList.push(newMsg);
52      }
53    }
54  }
55 }

```

```

52
53     if (msgList.length > 0) {
54         return [msgList];
55     } else {
56         return null;
57     }
58
59     } catch (e) {
60         node.error("Errore parsing: " + e.message);
61         return null;
62     }
63 } else {
64     return null;
65 }

```

Listing 7: Filter Elements and To CSV

Firstly, we check if the command string contains at least one of those 3 strings, then we parse the command string in JSON after replacing all the ' char with the " char. Now we analyze the data in the command string, saving the sequence number and all the other information needed.

Then we analyze for each of the attributes the name, status, and both data type and data value, here we only keep well-formed attributes as stated on the weBeep forum.

Creating the payload in a way that we can directly save it in the filtered_elems.csv.

Also we save the chartValue and chartType so that we can create the plot of both the RMS Current and the RMS Voltage.

Here we used a [msgList] so that each message will be sent one at a time.

For creating the plot we have used a switch to separate the messages looking at the value of the msg.chartType, and then we used a change node to substitute the value of the payload with only the one of the msg.chartValue variable so that the chart node could create the plot.

This is an example of the resulting graphs.

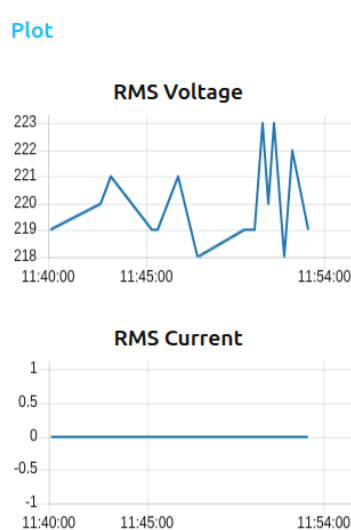


Figure 6: Graphs

The values of the RMS Current are always 0, while the RMS Voltage varies between 218 and 223 when present.

We have decided not to discard any messages even if they contain only partially correct data, assuming 0 when no value is present.

5 What to do? (3) 2 points

- If the message with Packet No. = N instead contains a Link Status message (Packet type = Link Status (0x08)), save the outgoing cost for each link associated with that ZigBee source address.
- If a later Link Status packet arrives from the same source, update the stored cost for each destination.
- At the end of the execution (which will be described later) save the cost for each link in a “outgoing_cost.csv” CSV file with the following fields: No.,Source,Destination,Cost
- Find the smallest source address in outgoing_cost.csv, treating the addresses as hexadecimal numbers.
- For that source address, sort its destination addresses in ascending hexadecimal order.
- Then send the outgoing cost for each destination address to your ThingSpeak channel using the HTTP API. For each message, put the outgoing cost in field1. Send the messages in destination-address order, and respect a rate limit of one message every 20 seconds.
- In all the other cases (Packet No = N not containing an ZBEE_ZCL Layer or a Link Status). Ignore the message!

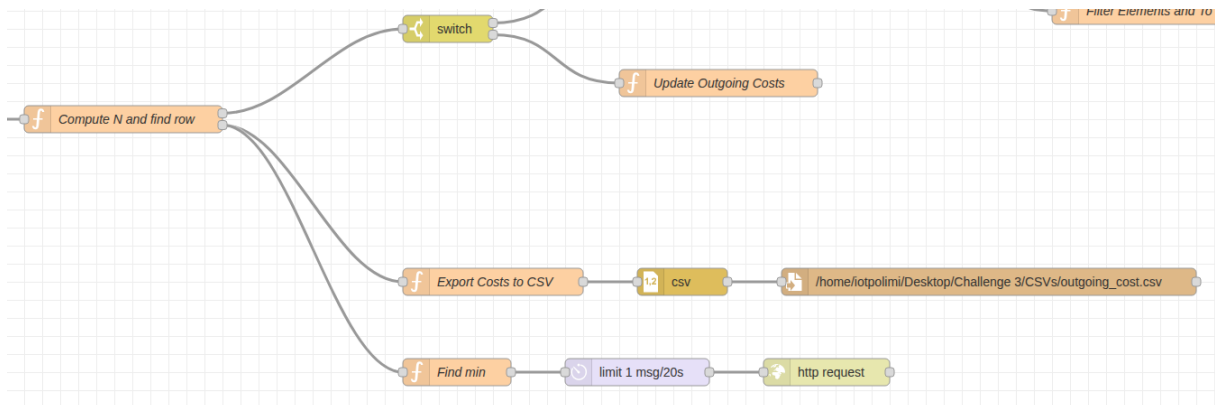


Figure 7: Link Status

When the `msg.csvRow["Command String"]` contains "Link Status", the message will go to the Update Outgoing Costs function node.

```
1 let row = msg.csvRow;
2 let cmdStr = row["Command String"];
3
4 let srcAddr = row["Source Address ZigBee"];
5
6 let linkCosts = flow.get("linkCosts") || {};
7
8 let data = JSON.parse(cmdStr.replace(/'/g, '"'));
9
10 let links = data["Links"] || [];
11
12 if(!linkCosts[srcAddr]){
13     linkCosts[srcAddr] = {};
14 }
15
16 for(let i = 0; i < links.length; i++){
17     let dst = links[i]["Address"];
18     let cost = links[i]["Outgoing Cost"];
19
20     linkCosts[srcAddr][dst] = cost;
21 }
22
23 flow.set("linkCosts", linkCosts);
24
25 return null;
```

Listing 8: Update Outgoing Costs

Here, we save the command string and parse it in the same way as before.

We initialize/read the `linkCosts` object saved in the flow, and we save all the links that arrived with this message.

Then, for each link, we update its value.

Lastly, we save the `linkCosts` object in the flow.

After the 200th message has arrived, we can save our outgoing_cost.csv file.

```
1 let linkCosts = flow.get("linkCosts");
2
3 if(!linkCosts){
4     return null;
5 }
6
7 let msgList = [];
8 let count = 1;
9
10 for(let src in linkCosts){
11     for (let dst in linkCosts[src]){
12         let newMsg = {
13             payload: {
14                 "No.": count,
15                 "Source": src,
16                 "Destination": dst,
17                 "Cost": linkCosts[src][dst]
18             }
19         };
20         msgList.push(newMsg);
21         count++;
22     }
23 }
24
25 return [msgList];
```

Listing 9: Export Costs to CSV

We go through each source saved in the linkCosts object, and for each of its destinations, we create the payload that we save in the csv file as we did with the previous .csv files.

Lastly we have the Find min function node that will find the smallest source address and sort its destination addresses before sending the value to the ThingSpeak channel.

```

1  let linkCosts = flow.get("linkCosts");
2
3  if(!linkCosts){
4      node.error("Nessun costo");
5      return null;
6  }
7
8  let minVal = Infinity;
9  let smallestSource = null;
10
11  for(let src in linkCosts){
12      let srcVal = parseInt(src, 16);
13      if(srcVal < minVal) {
14          minVal = srcVal;
15          smallestSource = src;
16      }
17  }
18
19  if (!smallestSource) {
20      return null;
21  }
22
23  let targetDsts = [];
24  for (let dst in linkCosts[smallestSource]) {
25      targetDsts.push({
26          dest: dst,
27          cost: linkCosts[smallestSource][dst]
28      });
29  }
30
31  targetDsts.sort((a, b) => parseInt(a.dest, 16) - parseInt(b.dest, 16));
32
33  let msgList = [];
34  for (let i = 0; i < targetDsts.length; i++) {
35      let newMsg = {
36          payload: {
37              api_key: "XJHNCROF5WDMH3E8",
38              field1: targetDsts[i].cost
39          }
40      };
41      msgList.push(newMsg);
42  }
43
44  return [msgList];

```

Listing 10: Find Min

We set the minimum value to infinity and the smallest source to null, then we go through each of the sources in the linkCosts object to find the one with the smallest address.

After finding the smallest one, we go through each of its destinations, saving the destination address and the cost.

After this, we can easily sort them with JavaScript and create the payload with the api key and the value for field1.

The messages are then sent to a delay node that will send 1 msg each 20s to an HTTP Request node.

6 Testing

For testing, we used different debug nodes during development and also used different terminals in listening mode to different topics so that we could analyze the correct formatting of the payload received at the Local Mosquitto broker.

For example

```
1 mosquitto_sub -h localhost -t "challenge3/id_generator" -p 1884
2
3 mosquitto_sub -h localhost -t "ZigBee/Coordinator" -p 1884
```

Listing 11: Testing

We also looked at the .csv files created by the program and analyzed the correct functionality of the program.

7 Deliveries

I terminated the program's execution the moment all queued messages were sent, resulting in an id_log.csv file containing around 700 rows.

For the filtered_elems.csv, we found 48 values.

While for the outgoing_cost.csv, we have 63 rows.